Ayan Ghanchi

Social Media System:

Introduction:

The Social Media System is a web application that allows users to create accounts, post photos and videos, and interact with posts through likes and comments. The main users are people who want to share content and interact with others. The backend will use spring boot, spring data JPA, an H2 database, DTOs, and a layered architecture with controller, service, repository, and domain layers.

Design:

Resources

User

Represents a registered user.

Attributes:

- id: Long
- username: String
- email: String
- displayName: String

Post

Represents content created by a user.

Attributes:

- id: Long
- content: String
- mediaUrl: String
- createdAt: LocalDateTime

Ayan Ghanchi

- author: User

Interaction

Represents a like or comment on a post.

Attributes:

- id: Long
- type: InteractionType
- commentText: String
- createdAt: LocalDateTime
- user: User
- post: Post

InteractionType contains:

- LIKE
- COMMENT

DTOs

Each resource will use request and response DTOs.

User

UserRequestDTO: username, email, displayName

UserResponseDTO: id, username, email, displayName

Post

PostRequestDTO: authorId, content, mediaUrl

PostResponseDTO: id, authorId, authorUsername, content, mediaUrl, createdAt

Interaction

Ayan Ghanchi

InteractionRequestDTO: userId, postId, type, commentText

InteractionResponseDTO: id, userId, username, postId, type, commentText, createdAt

UML Class Diagram

@startuml

class User {
  id: Long
  username: String
  email: String
  displayName: String
}

class Post {
  id: Long
  content: String
  mediaUrl: String
  createdAt: LocalDateTime
}

class Interaction {
  id: Long
  type: InteractionType

Ayan Ghanchi

```
  commentText: String
  createdAt: LocalDateTime
}

enum InteractionType {
  LIKE
  COMMENT
}

User "1" -- "0..*" Post : creates
User "1" -- "0..*" Interaction : performs
Post "1" -- "0..*" Interaction : receives

@enduml
```

Relationships

- **User to Post:** One-to-Many
- **User to Interaction:** One-to-Many
- **Post to Interaction:** One-to-Many

Each post belongs to one user.

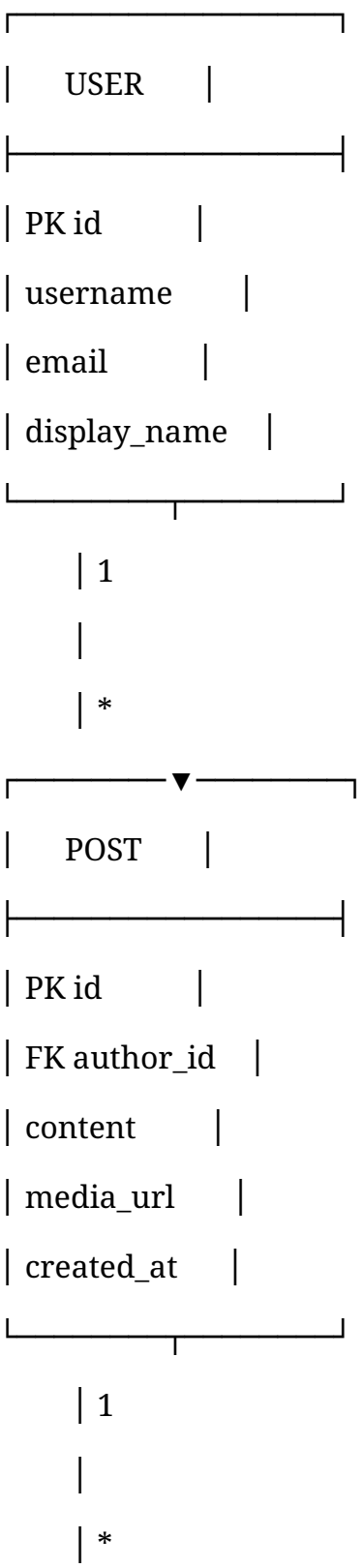
Each interaction belongs to one user and one post.

Collections will use FetchType.LAZY.

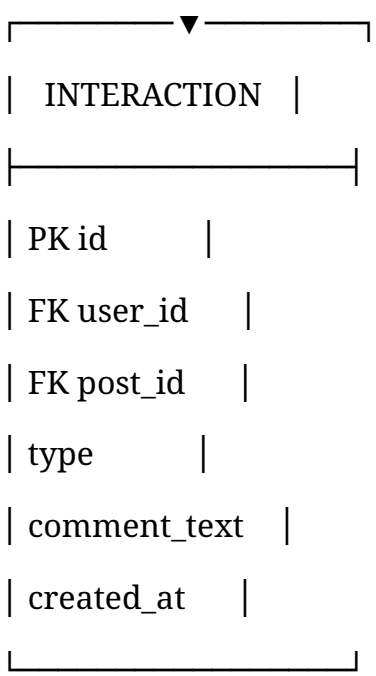
CascadeType.PERSIST can be used on one-to-many relationships.

Ayan Ghanchi

ERD / Database Schema:



Ayan Ghanchi



REST Endpoints:

Users

Method	Endpoint	Purpose
GET	/users	Get all users
GET	/users/{id}	Get one user
POST	/users	Create user
PUT	/users/{id}	Update user
DELETE	/users/{id}	Delete user
GET	/users/{id}/posts	Get a user's posts

Posts

Ayan Ghanchi

Method	Endpoint	Purpose
GET	/posts	Get all posts
GET	/posts/{id}	Get one post
POST	/posts	Create post
PUT	/posts/{id}	Update post
DELETE	/posts/{id}	Delete post
GET	/posts/{id}/interactions	Get interactions on a post

Interactions

Method	Endpoint	Purpose
GET	/interactions	Get all interactions
GET	/interactions/{id}	Get one interaction
POST	/interactions	Create interaction
PUT	/interactions/{id}	Update interaction
DELETE	/interactions/{id}	Delete interaction

Example Request and Response:

Create Post

Ayan Ghanchi

Request

POST /posts

```
{
  "authorId": 1,
  "content": "This is my first post!",
  "mediaUrl": "https://example.com/image.jpg"
}
```

Response

201 Created

```
{
  "id": 10,
  "authorId": 1,
  "authorUsername": "agspeedz",
  "content": "This is my first post!",
  "mediaUrl": "https://example.com/image.jpg",
  "createdAt": "2026-10-02T13:30:00"
}
```

Other expected status codes include:

- 200 OK for successful GET and PUT requests
- 201 Created for successful POST requests
- 204 No Content for successful DELETE requests
- 400 Bad Request for invalid input
- 404 Not Found when a resource does not exist

Ayan Ghanchi

Validation:

User

- username: @NotBlank
- email: @NotBlank, @Email
- displayName: @NotBlank

Post

- authorId: @NotNull, @Positive
- content: @NotBlank
- mediaUrl: optional

Interaction

- userId: @NotNull, @Positive
- postId: @NotNull, @Positive
- type: @NotNull
- commentText: required when type is COMMENT

Invalid requests will return:

400 Bad Request

Non-Functional Considerations

Error Handling

A NotFoundException will be used when a User, Post, or Interaction cannot be found.

Validation errors will be handled using MethodArgumentNotValidException.

Project Structure

Ayan Ghanchi

controller

service

repository

domain

dto

exception

The system will keep Controller, Service, Repository, and Domain responsibilities separate.

Testing

Later milestones will include basic Controller and Service tests.

Tests will verify:

- successful creation of resources
- successful retrieval
- invalid input
- resources that do not exist
- correct HTTP status codes